
A FRAMEWORK PROPOSAL • 2026

Your AI agents now
decide things inside a
runtime *you don't
own.*

Training opt-outs do not create runtime data sovereignty. Architecture has to.

THE PROBLEM

This is the *Custody Problem*.

When an enterprise runs its work through a managed agent, it isn't just buying model access.

It is delegating **context assembly, memory, tool execution, and policy defaults** to a runtime it did not build and cannot fully inspect.

THE SHIFT

The risk is not only *training data*.
It is *runtime usage control*.

The agent doesn't need to be retrained to leak your operations. It just needs to keep running.

Every retrieval, every cached answer, every tool invocation, every async job – each is a surface where operational knowledge accumulates inside someone else's environment.

FIVE RUNTIME SURFACES

Where operational knowledge quietly accumulates – even without model training.

SURFACE 01

Retrieval

Which documents the agent is allowed to see for this request.

SURFACE 02

Memory

What the agent remembers across turns, sessions, and users.

SURFACE 03

Tools

Which APIs and side-effects the agent can invoke on your behalf.

SURFACE 04

Cache

Where answers, embeddings, and intermediate results are reused.

SURFACE 05

Async jobs & telemetry

Background tasks and observability pipelines that quietly propagate context outside the original request boundary.

A FAILURE MODE

Silent context bleed.

A finance question pulls a fragment of an HR document into its reasoning. The answer looks fine. Nothing alerts. Nothing logs it as a violation.

The boundary was a configuration claim — not an architectural one.

THE CONCENTRATION RISK

0

Model-training cycles required for a third-party agentic runtime to become a strategic dependency you cannot leave.

The lock doesn't click on day one. It clicks the day you realize you can't migrate without rebuilding the reasoning your business now runs on.

THE DEEPER QUESTION

This is a data-sovereignty problem expressed as *information-flow control*.

Reasoning context, memory, and operational telemetry become derived data products inside execution environments the enterprise may not control.

By the time it manifests as an incident, the value has already migrated.

A PROPOSAL

Introducing *Hermetic Knowledge Isolation* — HKI.

An open framework for enforcing custody at every runtime surface: gateway, retrieval, memory, tools, cache, async jobs, telemetry.

Not a product. A contract any agentic platform can be built or audited against.

THE RUNTIME CONTRACT

Four properties any custody boundary must satisfy.

- 01 **One active domain per operation** signed before any retrieval, memory, or tool call begins.

- 02 **Labels preserved** through memory, cache, embeddings, async jobs, and telemetry – no surface drops them.

- 03 **Cross-domain work** uses scoped delegation and explicit release – never one mixed prompt or inferred global scope.

- 04 **Every boundary testable** through adversarial probes and verifiable release evidence across all runtime surfaces.

WHERE TO START

Audit one runtime operation *end-to-end.*

Ask: which domain is active, which child envelopes exist for multi-domain work, what release produced the briefing, and what breaks if scope is absent?

READ THE FULL PAPER · FOLLOW THE WORK

The Custody Problem in Enterprise Agentic AI — visual overview and the HKI v1 framework specification.

github.com/h3nok/HKI